

StockVision - Stock Market Analysis & Prediction Studio

README | Scilab GUIVerse Hackathon

I developed StockVision as an interactive Scilab desktop application for stock-market analysis, forecasting, model comparison, validation, visualization, and backtesting. The current version uses three forecasting approaches and connects them through one GUI workflow.

Core idea: I do not treat a prediction as the final answer. I let the user compare three modelling approaches, change meaningful parameters, observe evaluation metrics, validate models forward in time, and inspect a historical model-guided backtest against Buy & Hold.

1. Why I Built StockVision

I wanted to bridge the gap between an introductory calculator or plot and a complete interactive modelling workflow. A learner should be able to compare modelling philosophies, change the train/test split, inspect a moving-average trend, and see how a generated signal behaves after an explicit execution lag and trading costs.

I position StockVision primarily as a Finance and Business application with a strong Data Visualization and Analytics component.

2. What I Implemented

- Data: three bundled synthetic demo datasets plus custom CSV loading.
- Three forecasting models: Linear Regression, autoregressive (AR) time-series forecasting, and Exponential Smoothing using Holt's linear trend method.
- Interaction: model-picker dropdown, train/test split slider, AR lookback, BUY/SELL thresholds, transaction cost, slippage, and 20-day moving-average overlay.
- Evaluation: RMSE, MAE, MAPE, R-squared, and Prediction Accuracy percentage.
- Validation: 3-fold walk-forward validation for all three models.
- Prediction view: next-day estimate, colour-coded BUY/SELL/HOLD signal, rough residual-based uncertainty band, and actual-vs-predicted charts.
- Comparison: side-by-side metrics and a dedicated comparison bar chart.
- Backtesting: long/cash model-guided strategy versus Buy & Hold with a one-day execution lag and configurable costs.
- Risk: maximum drawdown, annualized volatility, Sharpe ratio, and per-trade win rate.
- Output: CSV, PDF, and TXT export plus reproducible sample outputs.

3. The Three Models

Linear Regression. I use lagged prices and derived indicators including moving averages, volatility, and daily return. Scaling parameters are calculated from the training split so test-period observations do not influence the scaler.

AR time-series model. I predict the next value from a configurable rolling window of previous values. It is a linear autoregressive model, not an LSTM or other deep-learning model.

Exponential Smoothing (Holt's linear trend method). I maintain a running level and trend estimate and forecast one step ahead as level plus trend. The implementation uses fixed level/trend smoothing parameters rather than fitting them from the data.

4. Validation, Correctness and Robustness

I keep time-series evaluation chronological and use multi-fold walk-forward validation across all three models. The implementation also validates dates, OHLCV structure, chronology,

duplicate dates, minimum data length, and numerical edge cases.

The current test suite contains 171 automated check calls in the submitted test file, excluding the helper function definition itself. It covers data validation, all three model paths, leakage/indexing regressions, signal generation, moving-average behaviour, backtesting, risk metrics, walk-forward validation, and edge cases. All 171 checks were actually executed (not just inspected) with `scilab-cli` and pass cleanly.

5. Backtesting

I use the fitted model's BUY/SELL/HOLD signal in a simple long-or-cash historical backtest. A signal can affect portfolio value only after an explicit one-day lag. The results are compared against Buy & Hold and include return and risk measures.

The execution model is deliberately simplified: it does not model bid/ask spread, partial fills, intraday movement, or market impact. Trades use the day's closing price. SELL exits an existing position; the application does not short.

6. GUI Design and Hackathon Alignment

I designed a purpose-built multi-panel GUI rather than copying a template. The interface uses substantially more than the minimum three distinct Scilab GUI components, including menus, popup menus, sliders, checkboxes, edit controls, push buttons, frames, results controls, message boxes, dedicated chart windows, and file dialogs.

The model picker is a single dropdown so the interface can scale beyond the current three models. Compare uses a dedicated comparison chart, while Run Analysis and Run Backtest use dedicated chart windows separate from the main GUI.

Hackathon emphasis	How I address it
Originality	Purpose-built StockVision GUI and modelling workflow; no tutorial/repository source copied.
Interactive problem solving	Major controls change real modelling, validation, signal, or backtest inputs.
Educational usefulness	Three-model comparison, Model Info, walk-forward validation, and visible metrics make model behaviour explorable.
Going beyond basics	Three fitted forecasting approaches plus validation, backtesting, and risk analysis.
Functionality & correctness	Separated modelling engine, validation safeguards, regression tests, and explicit execution lag.
Visualization	Actual-vs-predicted plots, moving-average overlay, backtest chart, and comparison bar chart.

7. Data and CSV Format

The bundled datasets are synthetic demonstration data, not live or downloaded historical prices. The application has no live market-data connection. Users can load their own CSV files.

Required columns:

Date,Open,High,Low,Close,Volume

Dates use YYYY-MM-DD. The loader validates the header, dates, numeric OHLCV values, OHLC consistency, non-negative volume, chronology, and minimum data length. Duplicate dates are rejected.

8. Running the Application

From PowerShell / Command Prompt:

```
scilab -f gui_app.sce
```

From an already-running Scilab console:

```
exec("gui_app.sce", -1);
```

No API keys or internet connection are required for the bundled offline datasets.

9. Project Structure

```
scilab_gui_app/  
|-- gui_app.sce  
|-- model_engine.sce  
|-- test_model_engine.sce  
|-- generate_sample_outputs.sce  
|-- MANUAL_TEST_CHECKLIST.md  
|-- sample_data/  
`-- outputs/
```

I keep the GUI layer and modelling logic separate: `gui_app.sce` handles the interface and callbacks, while `model_engine.sce` contains data validation, feature engineering, fitting, evaluation, signal generation, backtesting, and walk-forward validation for the three-model workflow.

10. Testing and Final Verification

I developed and tested the current application with Scilab 2026.1.0 on Windows. The automated suite is used for numerical/model correctness, while the manual checklist is used for live GUI behaviour.

11. Sample Outputs and Reproducibility

The submission package includes sample prediction, backtest, and model-comparison outputs, including a dedicated sample prediction output for Exponential Smoothing. `generate_sample_outputs.sce` provides the reproducible output workflow.

12. Submission Contents

The final package is organised into source code, documentation, demo video, problem statement, GUI screenshots, and sample outputs. The demo video is kept below two minutes.

13. Limitations

- Educational and analytical tool, not investment advice.
- Bundled data is synthetic demonstration data.
- No live market-data connection.

- Backtest execution assumptions are simplified.
- The strategy is long/cash only; there is no short selling.
- The uncertainty band is a rough residual-based normal approximation, not a rigorous probabilistic prediction interval.
- Exponential Smoothing uses fixed smoothing parameters rather than fitting alpha/beta from the data.
- Custom CSV files must follow the documented schema and date format.

14. References

I use the standard Scilab GUI and numerical capabilities available in the target Scilab environment. Any additional external algorithm or documentation reference used in the final project package should be listed explicitly here.